PHASE 8

PROJECT DEMONSTRATION

# AI WeatherWise API

## Intelligent Weather Forecasting, AI Insights and Recommendations Powered by Google Gemini AI

| | |
|---|---|
| **Project type** | AI-powered weather forecasting, weather insights and recommendation REST API / backend application |
| **Technology stack** | Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI · OpenWeatherMap API · Thunder Client / Postman |
| **Backend base URL** | http://localhost:5001 |
| **Document purpose** | Demonstration plan, demo video script, evidence checklist and video information |
| **Submitted by** | [Student Name] \| [Register Number] |
| **Department / College** | [Department Name] \| [College Name] |
| **Date** | [Submission Date] |

**Note**

This document is the plan and script for the demonstration. It contains no screenshots, API responses, tokens, keys, credentials, test results or video URL. Those must come from the working project after recording.

# About This Document

This Phase 8 document prepares the demonstration of **AI WeatherWise API**, a Node.js and Express.js REST API backend that provides secure user accounts, favorite location management, real-time weather information from OpenWeatherMap, and AI-generated weather summaries and recommendations from Google Gemini AI.

The structure follows the Phase 8 demonstration format: a demo plan, a scene-by-scene demo video script, an output evidence section, demo video information, a demonstration flow diagram, a presentation checklist and a conclusion. All content describes AI WeatherWise only.

## How to Use This Document

1. Read the Demo Plan (Section 1) and complete the Endpoint Verification Sheet using your actual project source.
2. Prepare your environment using the Before Recording checklist (Section 2.1).
3. Record the demo by following the scene script (Section 2.3) and adjusting the narration to what really appears on screen.
4. Capture the screenshots listed in Section 3 from the working project.
5. Upload the video to Google Drive and add the link in Section 4.
6. Complete the Presentation Checklist before submission.

## Notation Used

| Notation | Meaning |
|---|---|
| **[Square brackets]** | A value you must fill in from your own working project (for example an endpoint, a city name or a link). |
| **To be verified against the implemented project.** | The item could not be confirmed from the information available while preparing this plan. Check it against your source code before using it. |

## Contents


| | | |
|---|---|---|
| **1** | Demo Plan | Project name, purpose, key benefits, technology stack, objectives, endpoint verification sheet |
| **2** | Demo Video Script | Before recording, sequence summary and scene-by-scene script (Scenes 1–11) |
| **3** | Output Evidence | Screenshot placeholders and capture rules |
| **4** | Demo Video Information | Video details and Google Drive link placeholder |
| **–** | Flow Diagram, Checklist, Conclusion | Figure 1, presentation checklist and Phase 8 conclusion |

# 1. Demo Plan

## A. Project Name

> **AI WeatherWise – Intelligent Weather Forecasting, AI Insights and Recommendations Powered by Google Gemini AI**

## B. Project Purpose

AI WeatherWise is a REST API backend that provides weather-related services through a secure and modular API architecture. Users register and log in, and the server issues a JSON Web Token (JWT) that protects private routes. Each authenticated user can manage a personal list of favorite cities or locations, which is stored in MongoDB Atlas together with the user account data.

For weather data, the backend calls the OpenWeatherMap API and returns real-time weather information as a structured JSON response. For interpretation, the backend sends weather-related data to Google Gemini AI, which generates a weather summary and recommendation in plain language. Passwords are hashed with bcrypt.js, requests are validated, and errors are returned through centralized error handling. Secrets and API keys are kept in environment variables.

The API is demonstrated using Thunder Client or Postman, so no front-end interface is required. The demonstration covers only the features that are implemented in the project source.

## C. Key Benefits

| Benefit | Explanation |
|---|---|
| **Secure user authentication** | Users register and log in with credentials. Passwords are stored as bcrypt hashes and access is controlled with JWT. |
| **Favorite location management** | Authenticated users can keep a personal list of favorite cities or locations instead of typing them again and again. |
| **Real-time weather information** | Current weather data is retrieved from OpenWeatherMap and returned through the WeatherWise API. |
| **AI-powered weather interpretation** | Google Gemini AI turns weather data into an easy-to-read summary and recommendation. The wording can vary between requests. |
| **Structured REST API architecture** | Routes, middleware, controllers and services keep the code modular, and any client that can send HTTP requests can use the backend. |
| **Protected user-specific data** | Private routes require a valid JWT, and location data is associated with the authenticated user. |
| **Centralized validation and error handling** | Input is validated, and errors are returned as consistent JSON responses with suitable status codes. |

| Benefit | Explanation |
|---|---|
| **Cloud database integration** | User and location data is stored in MongoDB Atlas through Mongoose, so the data is persistent and accessible from the backend. |

## D. Technology Stack

| Technology | Role in AI WeatherWise |
|---|---|
| **Node.js** | Server-side JavaScript runtime that runs the backend. |
| **Express.js** | Web framework used for routes, middleware and controllers. |
| **MongoDB Atlas** | Cloud database that stores user accounts and favorite locations. |
| **Mongoose** | Defines schemas and models and communicates with MongoDB. |
| **JWT** | Token-based authentication for protected routes. |
| **bcrypt.js** | Hashes user passwords before they are stored. |
| **OpenWeatherMap API** | Provides real-time weather data. |
| **Google Gemini AI** | Generates AI-powered weather summaries and recommendations. |
| **dotenv** | Loads secrets and API keys from environment variables. |
| **CORS** | Controls which clients may call the API from a browser. |
| **Validation and error middleware** | Checks request input and returns centralized, consistent error responses. |
| **Thunder Client / Postman** | Sends API requests during the demonstration. |

## E. Demonstration Objectives

The demonstration shows the working WeatherWise system. Each objective below is included in the video only if the feature exists in the source code.

| # | Objective | What the audience will see |
|---|---|---|
| 1 | User registration | A new demo user is created through the registration API. |
| 2 | User login | The demo user logs in with email and password. |
| 3 | JWT authentication | A token is issued and used in the Authorization header. |
| 4 | Protected profile / API access | A protected route works only with a valid token. |
| 5 | Favorite location management | Locations are added and listed for the logged-in user (other operations only if implemented). |

| # | Objective | What the audience will see |
|---|---|---|
| 6 | Weather API request | A request for a city's weather is sent. |
| 7 | Real-time weather response | OpenWeatherMap data is returned as JSON. |
| 8 | AI weather summary / recommendation | Gemini AI generates a summary or recommendation. |
| 9 | Validation and error handling | One or more real error cases return proper error responses. |
| 10 | Final API output | A final successful response and a short wrap-up. |

## F. Endpoint Verification Sheet

Endpoints, request fields and response fields are not invented in this document. Before recording, fill in this sheet from your route files and controllers. Use the same values in the script and in the screenshots.

| Feature | Method | Endpoint | Auth required? |
|---|---|---|---|
| **User registration** | [ ] | To be verified against the implemented project. | [ ] |
| **User login** | [ ] | To be verified against the implemented project. | [ ] |
| **Protected profile / API** | [ ] | To be verified against the implemented project. | [ ] |
| **Add favorite location** | [ ] | To be verified against the implemented project. | [ ] |
| **List favorite locations** | [ ] | To be verified against the implemented project. | [ ] |
| **View / update / delete location** | [ ] | To be verified against the implemented project. (only if implemented) | [ ] |
| **Weather request** | [ ] | To be verified against the implemented project. | [ ] |
| **AI weather insight** | [ ] | To be verified against the implemented project. | [ ] |

# 2. Demo Video Script

**Target length:** about 3–5 minutes. The scene timings below add up to about 4 minutes 35 seconds when read at a steady pace. Timings are guides; shorten the narration if you run over. Use simple words and speak naturally.

## 2.1 Before Recording

Tick each item before you start recording.

| | Preparation item |
|---|---|
| ☐ | Start the WeatherWise backend. |
| ☐ | Confirm the MongoDB Atlas connection is established. |
| ☐ | Confirm the server is running on port 5001 (http://localhost:5001). |
| ☐ | Open Thunder Client / Postman. |
| ☐ | Prepare demo-only user details (a demo name, email and password). Never use a personal password. |
| ☐ | Prepare all required API requests in the order of the scenes, with a token variable for the JWT. |
| ☐ | Keep the architecture diagram ready to display. |
| ☐ | Keep the required test data ready (for example the city names for favorite locations and weather requests). |
| ☐ | Do NOT show the .env file. |
| ☐ | Do NOT show the MongoDB connection string. |
| ☐ | Do NOT show JWT secrets. |
| ☐ | Do NOT show the Gemini API key or the OpenWeatherMap API key. |
| ☐ | Do NOT expose passwords. |
| ☐ | Zoom Thunder Client / Postman so requests and responses are readable. |
| ☐ | Close unrelated windows and turn off notifications. |

**Security rule**

Never use personal passwords or real API keys visible on screen during the demo. If a token or key appears in a response or a header, crop, mask or blur it in the final video and screenshots.

### What to Hide and How

| Sensitive item | Where it may appear | How to handle it |
|---|---|---|
| **JWT token** | Login response, Authorization header, token variable | Mask or blur in the video and screenshots. Show only that a token exists. |
| **Passwords** | Registration and login request bodies | Use a demo-only password. Blur it if it is visible. |
| **MongoDB connection string** | .env file, server code | Never open the .env file or connection code on screen. |
| **Gemini API key / OpenWeatherMap API key** | .env file, service files | Do not open these files. Crop them out if they appear. |
| **JWT secret** | .env file, authentication code | Do not show it at any time. |

## 2.2 Demo Sequence Summary

| Scene | Demonstration | What is Shown | Time |
|---|---|---|---|
| **1** | Project Introduction | Project title, purpose and key benefits | 0:20 |
| **2** | Project Architecture | Architecture diagram and role of each layer | 0:30 |
| **3** | Server Startup and MongoDB Connection | Terminal with server on port 5001 and database connection | 0:15 |
| **4** | User Registration | Registration request, response and status code | 0:20 |
| **5** | User Login and JWT | Login request and token response (masked) | 0:25 |
| **6** | Protected API / Profile | Request with Bearer token, and access allowed | 0:20 |
| **7** | Favorite Location Management | Locations added and listed for the logged-in user | 0:40 |
| **8** | Weather API | Weather request and OpenWeatherMap response | 0:25 |
| **9** | Google Gemini AI Weather Insight | AI-generated summary / recommendation | 0:35 |
| **10** | Validation / Error Handling | One or more real error responses | 0:25 |
| **11** | Final Output and Conclusion | Final successful response and closing summary | 0:20 |

| Scene | Demonstration | What is Shown | Time |
|---|---|---|---|
| | **Total** | | **≈ 4:35** |

*If the implemented project needs a different order (for example, the weather request depends on a saved location), adjust the scenes but keep the total near 3–5 minutes.*

## 2.3 Scene-by-Scene Script

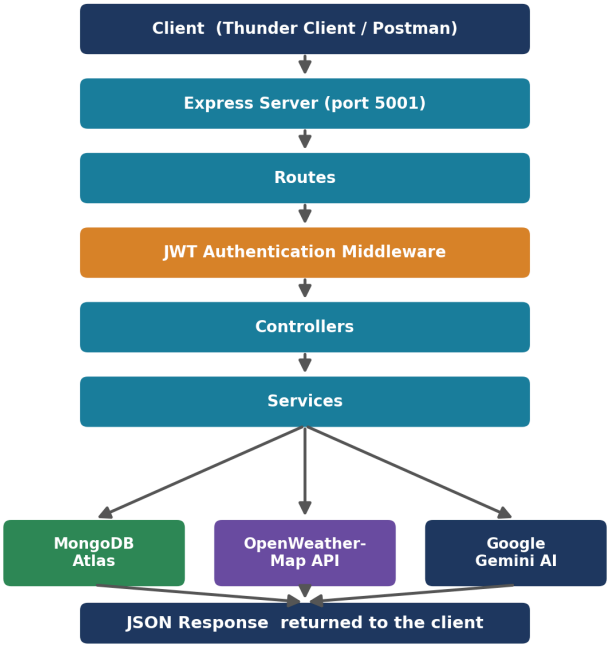
Read the narration in your own natural voice. Text inside [square brackets] must be replaced with the real value from your project. Say only what actually appears on screen.

### SCENE 1 – PROJECT INTRODUCTION (approx. 20 sec)

| | |
|---|---|
| **ON SCREEN** | Project title page or documentation cover. |
| **ACTION** | Show the project title. Keep the screen still while you speak. |
| **NARRATION** | *"Hello everyone. This is my project, AI WeatherWise API. It is a REST API backend that gives real-time weather information and AI-generated weather insights."*<br>*"Users can register, log in, save favorite locations and check the weather for a city. Weather data comes from OpenWeatherMap, and the AI summaries come from Google Gemini. Now I will show how the working system runs."* |
| **EXPECTED OUTPUT** | The title, project purpose and main technologies are clearly visible. |

### SCENE 2 – PROJECT ARCHITECTURE (approx. 30 sec)

| | |
|---|---|
| **ON SCREEN** | The architecture diagram (shown below). |
| **ACTION** | Show the diagram and point to each layer from top to bottom. |
| **NARRATION** | *"This is the architecture of AI WeatherWise. A request starts in Thunder Client and reaches the Express server. The routes send the request to the correct controller."*<br>*"For private routes, the JWT middleware checks the token first. The controller then calls a service. The services work with MongoDB for user and location data, with OpenWeatherMap for real-time weather, and with Google Gemini for AI insights."*<br>*"At the end, the server sends a JSON response back to the client."* |
| **EXPECTED OUTPUT** | The audience understands the request path and the role of each layer. |
| **NOTE** | Confirm that the layer names in the diagram match your project folders (routes, middleware, controllers, services). To be verified against the implemented project. |




*Architecture Diagram: AI WeatherWise – Request Processing Flow*

## Role of Each Layer

| Layer | Role in AI WeatherWise |
|---|---|
| **Express server** | Receives HTTP requests on port 5001 and manages the request–response cycle. |
| **Routes** | Map each URL and HTTP method to the correct controller. |
| **Authentication middleware** | Checks the JWT before a protected route can be used. |
| **Controllers** | Handle the request, call services and send the JSON response. |
| **Services** | Contain the logic for the database, OpenWeatherMap and Gemini AI. |
| **MongoDB** | Stores user accounts and favorite locations. |
| **OpenWeatherMap** | Provides real-time weather data. |
| **Gemini AI** | Generates the weather summary and recommendation. |

## SCENE 3 – SERVER STARTUP AND MONGODB CONNECTION (approx. 15 sec)

| | |
|---|---|
| **ON SCREEN** | Terminal or code editor terminal with the backend starting. |
| **ACTION** | Start the server (or show it already running). Show the address http://localhost:5001. |
| **NARRATION** | *"Now I will show the backend. Here is the terminal. The AI WeatherWise server is running on port 5001, so the base URL is http://localhost:5001. The connection to MongoDB is also established."* |

| | |
|---|---|
| **EXPECTED OUTPUT** | The terminal shows the startup messages produced by your actual project. Server startup output is shown based on the actual working project. |
| **NOTE** | Do not invent console messages. Show only what your terminal really prints, and make sure no connection string is visible. |

### SCENE 4 – USER REGISTRATION (approx. 20 sec)

| | |
|---|---|
| **ON SCREEN** | Thunder Client / Postman with the registration request. |
| **ACTION** | Show the HTTP method, the URL, the JSON request body with demo data, and click Send. Show the response and status code. |
| **NARRATION** | *"First, I register a new user. I send a POST request to the registration endpoint with the demo user details. The password here is only a demo password."*<br>*"The server validates the data, hashes the password with bcrypt.js and saves the user in MongoDB. The response shows the status code and the result. A new user account is now created."* |
| **EXPECTED OUTPUT** | A success response and status code from your working project. Use the actual endpoint: To be verified against the implemented project. |
| **NOTE** | Say "the password is not shown in the response" only if that is true on screen. Use the request fields required by your own registration route. |

### SCENE 5 – USER LOGIN AND JWT (approx. 25 sec)

| | |
|---|---|
| **ON SCREEN** | The login request and its response. |
| **ACTION** | Send the login request with the demo email and password. Show the response, then briefly show that the token is saved in the token variable. Mask the token. |
| **NARRATION** | *"Next, I log in with the same email and password. The server checks the credentials against the stored user. If they are correct, it returns a JWT token."*<br>*"This token proves who the user is, and it is required for all protected routes. For security, I am hiding the token value in this recording."* |
| **EXPECTED OUTPUT** | A successful login response containing a token (masked in the final video). Use the actual endpoint: To be verified against the implemented project. |
| **NOTE** | Do not show the real token clearly in the final recording. Mask or crop it. |

### SCENE 6 – PROTECTED API / PROFILE (approx. 20 sec)

| | |
|---|---|
| **ON SCREEN** | A protected request with the Authorization header. |
| **ACTION** | Show the header Authorization: Bearer <JWT> (token masked). Send the request and show the response. |

| | |
|---|---|
| **NARRATION** | *"Now I call a protected route. I send the token in the Authorization header as Bearer token. The server validates the JWT before it allows access."*<br>*"Because the token is valid, the request succeeds and I get the response for my account."* |
| **EXPECTED OUTPUT** | A successful response for the logged-in user. Use an actual protected endpoint: To be verified against the implemented project. |
| **NOTE** | The failure case without a token is shown later in Scene 10, so you do not need to show it here. |

## SCENE 7 – FAVORITE LOCATION MANAGEMENT (approx. 40 sec)

| | |
|---|---|
| **ON SCREEN** | Thunder Client / Postman with the Location API requests. |
| **ACTION** | Using the token, add a favorite city or location, then list the user's favorite locations. Show each request, response and status code. Continue with the other operations only if they exist in the code. |
| **NARRATION** | *"Now I will manage favorite locations. Every request uses my JWT, so the locations belong to my account. First, I add a favorite city with a POST request. The server saves it in MongoDB and links it to my user."*<br>*"Next, I get my list of favorite locations. The new city appears in the response. Other users cannot see this list, because it is connected to my account."* |
| **OPTIONAL LINES** | Use these only for operations that actually exist in the code:<br>View: "Now I open one location using its ID."<br>Update: "Now I update the location details, and the response shows the change."<br>Delete: "Finally, I delete one location, and it no longer appears in the list." |
| **EXPECTED OUTPUT** | The saved location appears in the list for the logged-in user. Endpoints and fields: To be verified against the implemented project. |
| **NOTE** | Plan two demo locations. If you show delete, delete only one, so a saved location remains for the next scenes. |

## SCENE 8 – WEATHER API (approx. 25 sec)

| | |
|---|---|
| **ON SCREEN** | The weather request and response in Thunder Client / Postman. |
| **ACTION** | Show the HTTP method, URL and input (for example a city name, if that is how your endpoint works). Send the request and scroll through the response. |
| **NARRATION** | *"Now I request the weather. I send the city as input to the weather endpoint. The server calls the OpenWeatherMap API and returns real-time weather information as JSON."* |

| | |
|---|---|
| | *"In the response, I can see the weather details for [city name], such as [mention only the fields you actually see on screen]."* |
| **EXPECTED OUTPUT** | A real-time weather response from OpenWeatherMap through your API. Use the actual response of your project. Do not describe any field that is not visible on screen. |
| **NOTE** | Weather values change over time, so the numbers in the video are the values at recording time. Endpoint: To be verified against the implemented project. |

## SCENE 9 – GEMINI AI WEATHER INSIGHT (approx. 35 sec)

| | |
|---|---|
| **ON SCREEN** | The AI weather insight request and response. |
| **ACTION** | Show the method, URL and input. Send the request, wait for the response and scroll through the generated text. |
| **NARRATION** | *"Now I use the AI feature. I send weather-related input to the AI endpoint. The AI service sends this to the Google Gemini API, and Gemini generates a weather summary and recommendation."*<br>*"Here is the generated result. Because this text is created by AI, the wording can be different each time I send the request. The AI gives easy advice based on the weather data."* |
| **EXPECTED OUTPUT** | An AI-generated weather summary and/or recommendation in the JSON response. Endpoint and input: To be verified against the implemented project. |
| **NOTE** | Do not show or describe the prompt text unless it is visible in your own request. Do not claim the output is always the same. The AI call may take a few seconds, so do not talk over the silence: cut it in editing if needed. |

## SCENE 10 – VALIDATION / ERROR HANDLING (approx. 25 sec)

| | |
|---|---|
| **ON SCREEN** | One or more real error responses in Thunder Client / Postman. |
| **ACTION** | Send one or two invalid requests and show the response body and status code. Choose from the cases in the table below, only if your project supports them. |
| **NARRATION** | *"Finally, I show error handling. First, I call a protected route without a token, and the server rejects the request. Next, I send invalid input, and the API returns a validation error."*<br>*"These errors are handled by the API's validation and centralized error handling, so the client always gets a clear JSON response with a proper status code."* |
| **EXPECTED OUTPUT** | The actual error responses of your project. Do not fake error responses. |

| Error case | Safe way to show it | Include? |
|---|---|---|
| **Missing JWT** | Call a protected route with no Authorization header. | ☐ |
| **Invalid token** | Use an obviously wrong token value (never a real one). | ☐ |
| **Invalid input** | Send a request with a required field missing. | ☐ |
| **Invalid location** | Use a city name that does not exist, if your API handles it. | ☐ |
| **External API failure** | Show only if it happens naturally. Never change or expose the API key to force it. | ☐ |
| **AI service failure** | Show only if it happens naturally. Never change or expose the API key to force it. | ☐ |

## SCENE 11 – FINAL OUTPUT AND CONCLUSION (approx. 20 sec)

| | |
|---|---|
| **ON SCREEN** | A final successful API response, followed by the project title. |
| **ACTION** | Show the last successful response (for example the AI weather insight or the favorite location list). Return to the title slide. |
| **NARRATION** | *"In this demo, I showed user registration and login with JWT, protected routes, favorite location management, real-time weather from OpenWeatherMap, and AI weather insights from Google Gemini."*<br>*"I also showed validation and error handling. All responses are returned as JSON from the backend running on port 5001. Thank you for watching."* |
| **EXPECTED OUTPUT** | A clear final response and a short closing message. |
| **NOTE** | Mention only the features you actually demonstrated in Scenes 4–10. |

# 3. Demo Video Information

| Item | Details |
|---|---|
| **Project Name** | AI WeatherWise – Intelligent Weather Forecasting, AI Insights and Recommendations Powered by Google Gemini AI |
| **Demo Video Platform** | Google Drive |
| **Video Link** | [Google Drive Demo Video Link – To be added] |
| **Access** | Anyone with the link – Viewer |
| **Video Duration** | Approximately 3–5 minutes |
| **Video Contents** | • Project introduction<br>• Project purpose<br>• Key benefits<br>• Technology stack<br>• Architecture<br>• Working process<br>• Authentication<br>• Favorite locations<br>• Weather API<br>• OpenWeatherMap integration<br>• Gemini AI weather insight<br>• Validation / error handling<br>• Final API output |

**Before sharing the link**

Set the Google Drive sharing option to “Anyone with the link – Viewer”, then open the link in a private/incognito window to confirm the video plays. The link is a placeholder until the actual video is uploaded.

## Demonstration Flow Diagram

The demonstration follows the sequence below. Read left to right on row 1, right to left on row 2, and left to right on row 3.

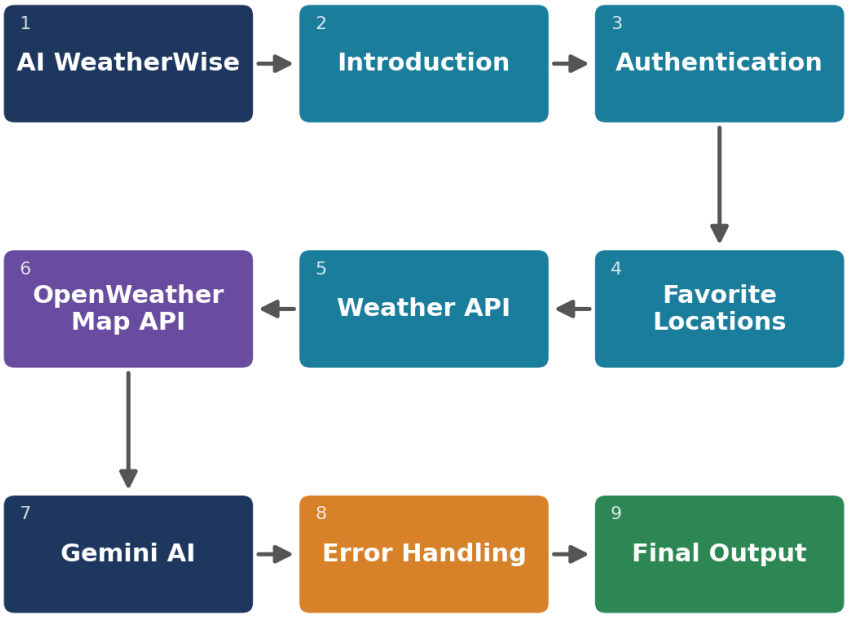



*Figure 1: AI WeatherWise – Project Demonstration Flow*

## Presentation Checklist

Complete this checklist before recording and before submission.

| | Item | Done |
|---|---|---|
| ☐ | Server running on port 5001 | |
| ☐ | MongoDB connection working | |
| ☐ | Weather API working | |
| ☐ | Gemini AI integration working | |
| ☐ | Demo user prepared | |
| ☐ | Thunder Client / Postman requests prepared | |
| ☐ | JWT environment variable configured | |
| ☐ | Architecture diagram ready | |
| ☐ | Screenshots captured | |
| ☐ | .env hidden | |
| ☐ | API keys hidden | |
| ☐ | Passwords hidden | |
| ☐ | JWT hidden | |
| ☐ | MongoDB credentials hidden | |
| ☐ | Final API output ready | |
| ☐ | Demo video recorded | |
| ☐ | Video uploaded to Google Drive | |
| ☐ | Video link added to documentation | |

# Phase 8 Conclusion

The Phase 8 demonstration presents the working AI WeatherWise system. It follows the process from user registration and login to the use of JWT-protected routes, and then shows how authenticated users manage their favorite locations, retrieve real-time weather information through the OpenWeatherMap integration, and receive weather summaries and recommendations generated by Google Gemini AI.

The demonstration also covers how validation and centralized error handling protect the API, and it ends with the final JSON API output returned by the backend running on http://localhost:5001.

The plan, video script, evidence checklist and video information in this document are prepared so that the demonstration is complete and organized. Screenshots and video evidence must come from the actual working project. Results, endpoints, responses and AI output should be described only after they have been demonstrated on screen. The demo video link and the screenshots are to be added after recording.

> **Final reminder**
> Before submitting, confirm that every endpoint, request field and response shown in your evidence matches the implemented project, and that no secret, token or API key is visible anywhere.